



Министерство науки и высшего образования
Российской Федерации

Федеральное государственное автономное
образовательное учреждение высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана (национальный
исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

Факультет «Фундаментальные науки»

Кафедра «Математическое моделирование»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к выпускной квалификационной работе

на тему:

ЭВРИСТИЧЕСКИЕ И ML-ПОДХОДЫ К РЕШЕНИЮ ЗАДАЧИ 3D BIN PACKING

Студент группы ФН12-81Б

(подпись, дата) *И.В. Иртюга*

Руководитель ВКР

(подпись, дата) *Д.А. Фетисов*

Нормоконтролер

(подпись, дата) *М.А. Велищанский*

РЕФЕРАТ

Расчетно-пояснительная записка содержит 45 с., 8 рис., 9 табл., 11 источников, 2 приложения.

Ключевые слова: 3D BIN PACKING, ПАЛЛЕТИЗАЦИЯ, SMART 3D PACKING, ЭВРИСТИЧЕСКИЕ АЛГОРИТМЫ, LARGE NEIGHBORHOOD SEARCH, ГЕНЕТИЧЕСКИЙ АЛГОРИТМ, GAN, MAXRECTS, ЖАДНЫЙ АЛГОРИТМ, ВАЛИДАЦИЯ УКЛАДКИ.

Объектом исследования является программная система построения трехмерной укладки коробов на паллету. Предметом исследования являются эвристические, метаэвристические и ML-ориентированные методы решения задачи 3D Bin Packing в постановке Smart 3D Packing.

Цель выпускной квалификационной работы состоит в исследовании и сравнении реализованных подходов к решению задачи 3D Bin Packing с учетом прикладных ограничений складской логистики: габаритов паллеты, грузоподъемности, допустимых ориентаций, опоры, хрупкости товаров и времени расчета.

В работе рассмотрены постановка задачи, структура входных и выходных данных, метрика качества, генерация синтетических наборов данных, архитектура проекта и реализованные решатели. Подробно описаны жадный алгоритм на extreme points, послойный алгоритм на основе MaxRects, метаэвристика Large Neighborhood Search, гибридный метод GAN+GA, точный и ограниченный перебор для малых задач, а также многопаллетное расширение. Экспериментальная часть основана на сохраненных результатах проекта и едином валидаторе, что обеспечивает сопоставимость методов.

По результатам анализа установлено, что в single-pallet постановке на общих наборах данных наилучший средний итоговый score показывает LNS: 0.7111 на наборе из 100 задач, 0.7434 на наборе из 200 задач и 0.7298 на контрольном наборе. Преимущество LNS объясняется лучшим балансом между утилизацией объема, полнотой размещения, соблюдением ограничений хрупкости и временем выполнения.

СОДЕРЖАНИЕ

РЕФЕРАТ	2
ВВЕДЕНИЕ	5
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ПОСТАНОВКА ЗАДАЧИ . .	7
1.1 Предметная область	7
1.2 Входные данные	8
1.3 Выходные данные	9
1.4 Жесткие ограничения	9
1.5 Метрика качества	10
1.6 Критерии качества упаковки	11
1.7 Выводы по разделу	12
2 ОБЗОР МЕТОДОВ РЕШЕНИЯ ЗАДАЧИ 3D BIN PACKING	13
2.1 Классические эвристики	13
2.2 Extreme Points и Bottom-Left-Fill	14
2.3 MaxRects и послойная укладка	15
2.4 Large Neighborhood Search	16
2.5 Генетические алгоритмы и ML-подходы	17
2.6 Точный перебор и branch-and-bound	18
2.7 Выводы по разделу	19
3 РАЗРАБОТАННОЕ РЕШЕНИЕ SMART 3D PACKING	21
3.1 Структура репозитория	21
3.2 Генерация данных	22
3.3 Валидатор и единая оценка	23
3.4 Greedy-решатель	23
3.5 MaxRects-решатель	24
3.6 LNS-решатель	24
3.7 GAN+GA-решатель	26
3.8 Визуализация	26
3.9 Выводы по разделу	28

4	ЭКСПЕРИМЕНТЫ, МЕТРИКИ И АНАЛИЗ РЕЗУЛЬТАТОВ	29
4.1	Методика эксперимента	29
4.2	Сводные результаты	30
4.3	Компоненты метрики	31
4.4	Время выполнения	33
4.5	Сравнение LNS и Greedy	34
4.6	Сравнение с brute force на малых задачах	35
4.7	Многопаллетное расширение	36
4.8	Анализ ошибок и ограничений	36
4.9	Направления развития	37
4.10	Выводы по разделу	38
	ЗАКЛЮЧЕНИЕ	39
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	42
	ПРИЛОЖЕНИЕ А	44
	ПРИЛОЖЕНИЕ Б	45

ВВЕДЕНИЕ

Задача упаковки объектов в ограниченный контейнер является одной из классических задач комбинаторной оптимизации. В трехмерном варианте требуется разместить набор прямоугольных параллелепипедов внутри контейнера или на паллете так, чтобы объекты не пересекались и не выходили за заданные границы. В прикладной логистике такая задача возникает при сборке заказов, паллетизации товаров, загрузке транспорта и планировании складских операций.

В крупном ритейле качество паллетизации напрямую влияет на стоимость перевозки и устойчивость товарного потока. Недостаточно плотная укладка приводит к перевозке незаполненного объема, а неучет веса, хрупкости и устойчивости увеличивает риск повреждения товаров. Поэтому практическая постановка 3D Bin Packing существенно шире чисто геометрической задачи: необходимо одновременно учитывать размеры, массу, допустимые ориентации, опору снизу, возможность штабелирования и время расчета.

Тема настоящей выпускной квалификационной работы связана с проектом Smart 3D Packing. Условие проекта формулирует задачу построения алгоритмического модуля 3D Pallet Packing Optimizer, который по описанию паллеты и списка коробов возвращает план размещения с координатами и ориентациями каждого короба. Согласно условию, решение проходит двухэтапную проверку: сначала контролируются жесткие ограничения физической корректности, затем вычисляется взвешенная метрика качества.

Актуальность работы обусловлена тем, что задача 3D Bin Packing относится к NP-трудным задачам [1], а практически значимые экземпляры содержат десятки и сотни объектов. Полный перебор быстро становится неприменимым, поэтому в реальных системах используются эвристики, локальный поиск, метаэвристики и гибридные методы [2, 3]. В исследуемом проекте реализовано несколько таких подходов, что позволяет сравнить их в едином экспериментальном контуре.

Цель работы – исследовать эвристические и ML-подходы к решению задачи 3D Bin Packing в проекте Smart 3D Packing и определить, какие методы обеспечивают лучший баланс качества укладки и времени расчета.

Для достижения цели решаются следующие задачи:

1. Изучить условие задачи Smart 3D Packing, требования к входным и вы-

ходным данным и правила валидации;

2. Формализовать ограничения и метрику качества упаковки;
3. Проанализировать структуру репозитория и восстановить pipeline решения;
4. Описать генерацию синтетических датасетов и состав используемых наборов данных;
5. Рассмотреть реализованные алгоритмы: greedy, MaxRects, LNS, GAN+GA, brute force и многопаллетное расширение;
6. Провести сравнительный анализ сохраненных экспериментальных результатов;
7. Сформулировать ограничения текущей реализации и направления дальнейшего развития.

Объект исследования – алгоритмическая система трехмерной укладки коробов на паллету.

Предмет исследования – эвристические, метаэвристические и ML-ориентированные методы построения допустимой и качественной укладки в задаче 3D Bin Packing.

Практическая значимость работы заключается в том, что исследуемая система объединяет генератор данных, набор решателей, валидатор, средства визуализации и сохраненные benchmark-результаты. Такая структура может быть использована как основа для дальнейших исследований алгоритмов паллетизации и для прототипирования прикладного логистического модуля.

Работа состоит из введения, четырех разделов основной части, заключения, списка использованных источников и приложений. В первом разделе рассматриваются предметная область и постановка задачи. Во втором разделе приведен обзор методов решения 3D Bin Packing. В третьем разделе описана архитектура проекта Smart 3D Packing и реализация решателей. В четвертом разделе представлены датасеты, эксперименты, метрики и интерпретация результатов.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ПОСТАНОВКА ЗАДАЧИ

1.1 Предметная область

Паллетизация в складской логистике состоит в формировании устойчивой грузовой единицы из множества товарных коробов. На практике решение должно быть не только плотным, но и выполнимым для транспортировки: тяжелые товары не должны разрушать хрупкие, нестабильные слои не должны приводить к падению груза, а ограничения по высоте и массе должны соблюдаться для совместимости со складским оборудованием и транспортом.

В условии Smart 3D Packing отдельно выделены типичные проблемы ручной или простой эвристической сборки: низкая утилизация объема, риск повреждений из-за неправильного распределения веса и нестабильность конструкции. Эти факторы показывают, что задача имеет многокритериальный характер. Один только максимум занятого объема не гарантирует хорошего решения, если оно нарушает устойчивость, портит товар или требует слишком долгого расчета.

С математической точки зрения задача относится к семейству bin packing и cutting/packing problems [4]. В классической трехмерной постановке требуется разместить прямоугольные объекты в одном или нескольких контейнерах. Даже более простые варианты bin packing являются вычислительно сложными, а добавление трехмерной геометрии и физических ограничений резко увеличивает размер пространства решений [1, 6].

В прикладной складской системе решение задачи упаковки обычно является частью более широкого процесса. До запуска алгоритма уже известен состав заказа, тип паллеты, допустимая высота укладки и ограничения по обработке товара. После построения укладки результат должен быть понятен оператору или следующему программному модулю: необходимо знать, какие короба размещены, какие остались вне паллеты и по какой причине. Поэтому в данной работе постановка рассматривается не как изолированная оптимизационная формула, а как вычислительный pipeline с входными данными, решателем, валидатором и отчетом о качестве.

Особенность задачи состоит в сочетании дискретных и непрерывных решений. С одной стороны, требуется выбрать порядок коробов и ориентации, то

есть работать с перестановками и конечными множествами поворотов. С другой стороны, результат задается координатами в трехмерном пространстве, и малое изменение координаты может изменить опору, пересечения или доступный объем. Из-за этого метод должен одновременно быть достаточно быстрым для перебора вариантов и достаточно строгим при геометрической проверке.

1.2 Входные данные

Вход одной задачи представлен JSON-объектом. Он содержит описание паллеты и список типов коробов. Для паллеты задаются:

- Длина `length_mm`;
- Ширина `width_mm`;
- Максимальная высота укладки `max_height_mm`;
- Максимальная масса `max_weight_kg`.

Для каждого типа короба задаются идентификатор SKU, описание, габариты, масса, количество экземпляров и дополнительные признаки:

- `strict_upright` – запрет поворота вокруг горизонтальных осей;
- `fragile` – признак хрупкости;
- `stackable` – возможность ставить другие короба поверх данного.

В генераторе проекта используются типовые архетипы товаров фудритейла: вода, сахар, вино, бананы, чипсы, яйца и консервы. Размеры и массы этих архетипов варьируются небольшим случайным шумом, что позволяет получать воспроизводимые, но не полностью одинаковые задачи.

Для алгоритмов важно, что входной JSON хранит не отдельный плоский список коробов, а типы SKU с количеством экземпляров. На этапе решения эти количества разворачиваются в отдельные объекты, поскольку каждый экземпляр получает собственные координаты и ориентацию. Такой формат ближе к реальным заказам: в них часто встречается несколько коробов одного типа, но размещаться они могут в разных слоях и в разных ориентациях.

Дополнительные признаки SKU задают логистический смысл задачи. Например, `strict_upright` ограничивает множество допустимых поворотов, `fragile` влияет на штраф за размещение тяжелых товаров сверху, а `stackable` определяет возможность использовать короб как надежную опору. Эти признаки делают задачу более содержательной, чем чистая упаковка

прямоугольников, и позволяют сравнивать алгоритмы по практической пригодности укладки.

1.3 Выходные данные

Результатом работы решателя является JSON-ответ, содержащий идентификатор задачи, версию решателя, время расчета и список размещенных коробов. Для каждого размещенного экземпляра указываются:

- `sku_id` и `instance_index`;
- Координаты нижнего ближнего левого угла `x_mm`, `y_mm`, `z_mm`;
- Фактические размеры после поворота;
- Код ориентации `rotation_code`.

Если часть объектов не размещена, она фиксируется в массиве `unplaced` с указанием количества и причины: превышение массы, высоты или отсутствие места. Такая структура удобна для последующей валидации и анализа: ответ содержит как геометрию решения, так и информацию о неполноте размещения заказа.

Разделение размещенных и неразмещенных объектов важно для честного сравнения методов. Алгоритм не должен получать высокий результат только потому, что разместил небольшую часть удобных коробов и проигнорировал остальные. Поэтому полнота размещения входит в итоговую метрику, а массив `unplaced` позволяет отдельно анализировать причины отказов. Если большинство отказов связано с массой, дальнейшее улучшение геометрической эвристики почти не поможет. Если же отказы вызваны отсутствием места, имеет смысл улучшать порядок размещения, восстановление свободных областей и работу с ориентациями.

1.4 Жесткие ограничения

В проекте реализован валидатор `validator.py`, который проверяет физическую корректность решения. Нарушение любого жесткого ограничения делает решение невалидным для конкретной задачи.

Основные ограничения следующие:

1. Каждый размещенный короб должен целиком находиться внутри габаритов паллеты;

2. Объемы любых двух коробов не должны пересекаться;
3. Суммарная масса размещенных товаров не должна превышать грузоподъемность паллеты;
4. Если `strict_upright=true`, фактическая высота короба должна совпадать с исходной высотой;
5. Короб, расположенный выше пола паллеты, должен иметь опору снизу не менее чем на 60% площади основания согласно формуле (1);
6. Фактические размеры размещенного короба должны быть перестановкой исходных размеров, что исключает некорректное изменение габаритов.

Условие опоры записывается следующим образом:

$$\frac{S_{\text{support}}}{S_{\text{base}}} \geq 0.6, \quad (1)$$

где S_{base} – площадь основания проверяемого короба, а S_{support} – суммарная площадь пересечения его основания с верхними гранями коробов, лежащих непосредственно ниже.

1.5 Метрика качества

Если решение прошло жесткие проверки, валидатор вычисляет итоговую метрику по формуле (2):

$$\text{score} = 0.50U_{\text{vol}} + 0.30C_{\text{items}} + 0.10F_{\text{frag}} + 0.10T_{\text{time}}. \quad (2)$$

Компонента утилизации объема рассчитывается по формуле (3):

$$U_{\text{vol}} = \frac{\sum_{i \in P} l_i w_i h_i}{LWH}, \quad (3)$$

где P – множество размещенных коробов, l_i, w_i, h_i – их фактические размеры, L, W, H – габариты паллеты.

Полнота заказа определяется по формуле (4):

$$C_{\text{items}} = \frac{N_{\text{placed}}}{N_{\text{total}}}, \quad (4)$$

где N_{placed} – число размещенных экземпляров, N_{total} – общее число экземпляров во входной задаче.

Оценка хрупкости задается формулой (5):

$$F_{\text{frag}} = \max(0, 1 - 0.05n_{\text{viol}}), \quad (5)$$

где n_{viol} – число случаев, когда короб массой более 2 кг расположен непосредственно над хрупким коробом с ненулевым перекрытием в плоскости XU .

Временной балл имеет ступенчатый вид: при времени до 1 секунды он равен 1.0, при времени от 1 до 5 секунд – 0.7, при времени от 5 до 30 секунд – 0.3, при большем времени – 0.0. Такая метрика поощряет алгоритмы, которые дают хороший результат в режиме, близком к реальному времени.

Выбор весов в метрике отражает приоритеты задачи. Наибольший вес получает утилизация объема, поскольку она напрямую связана с эффективностью использования паллеты и транспорта. Полнота размещения имеет второй по величине вес: с практической точки зрения важно не только заполнить объем, но и обработать как можно большую часть заказа. Компоненты хрупкости и времени имеют меньший вес, но не являются декоративными: они предотвращают ситуации, когда плотная укладка достигается ценой повреждения товара или чрезмерно долгого расчета.

Важно, что метрика применяется только после проверки жестких ограничений. Если решение выходит за границы паллеты, содержит пересечения или нарушает правило опоры, оно не должно сравниваться с валидными решениями по среднему score. Такой порядок соответствует инженерной логике: сначала проверяется физическая допустимость, затем оценивается качество среди допустимых вариантов.

1.6 Критерии качества упаковки

Формула (2) показывает, что 80% итогового score формируют плотность упаковки и доля размещенных товаров. Однако оставшиеся 20% также существенны, поскольку они отражают практическую применимость решения. Метод, который достигает высокой плотности за счет нарушения хрупкости или слишком большого времени расчета, может уступить более сбалансированному подходу.

Таким образом, в работе используются следующие критерии сравнения:

- Средний итоговый score на наборе задач;

- Средняя утилизация объема;
- Средняя полнота размещения;
- Средний балл хрупкости;
- Средний временной балл и фактическое время решения;
- Число валидных решений.

1.7 Выводы по разделу

Задача Smart 3D Packing является прикладной модификацией 3D Bin Packing, в которой геометрическая упаковка дополняется логистическими ограничениями. Наиболее важными особенностями постановки являются поддержка поворотов, проверка опоры, учет массы, запрет некорректного размещения тяжелых товаров над хрупкими и ограничение времени. Это делает задачу удобной для сравнения как быстрых эвристик, так и более сложных метаэвристических и ML-подходов.

2 ОБЗОР МЕТОДОВ РЕШЕНИЯ ЗАДАЧИ 3D BIN PACKING

2.1 Классические эвристики

Для задач упаковки часто применяются эвристики последовательного размещения: First Fit, Best Fit, First Fit Decreasing и их двумерные или трехмерные аналоги [5, 6]. Общая идея таких методов состоит в том, что объекты упорядочиваются по некоторому приоритету, после чего каждый объект размещается в первую или лучшую допустимую позицию.

В прикладной задаче паллетизации порядок размещения становится самостоятельным параметром качества. Сортировка только по объему обычно повышает плотность нижних слоев, но может ухудшать обращение с хрупкими товарами. Сортировка по массе помогает удерживать тяжелые товары внизу, однако иногда приводит к тому, что легкие, но крупные коробки оказываются неразмещенными. Поэтому практические эвристики часто используют составной приоритет: объем, массу, высоту, хрупкость, разрешенные ориентации и наличие запрета на штабелирование.

Отличие логистической постановки от абстрактной задачи упаковки состоит в том, что допустимое решение должно быть удобно не только для математической модели, но и для последующей перевозки. В задачах загрузки контейнеров и совмещенного планирования маршрутов также подчеркивается, что геометрическая плотность должна рассматриваться совместно с ограничениями реального обращения с грузом [8]. Для рассматриваемого проекта это означает, что метод не может оцениваться только по заполненному объему: важны полнота размещения, устойчивость, хрупкость и время расчета.

В трехмерной постановке важную роль играют кандидатные позиции. Полный перебор всех координат невозможен, поэтому обычно используются специальные множества точек: углы свободных областей, extreme points, границы уровней или прямоугольники свободного пространства. Такие методы дают быстрое допустимое решение и хорошо подходят как baseline.

Недостаток последовательных эвристик состоит в их локальности. Если ранний выбор оказался неудачным, алгоритм обычно не возвращается к нему и не перестраивает уже созданную укладку. В задачах паллетизации это особенно заметно: один крупный короб может заблокировать полезную область, а

хрупкий или нештабелируемый товар на нижнем уровне может испортить всю структуру над собой.

Несмотря на это, классические эвристики остаются необходимой частью экспериментального контура. Во-первых, они дают понятный baseline, с которым можно сравнивать более сложные методы. Во-вторых, быстрые эвристики часто используются как начальная фаза для метаэвристик: хорошее стартовое решение уменьшает объем последующего поиска. В-третьих, простые методы позволяют быстро выявлять ошибки в валидаторе и формате данных, так как их поведение легко интерпретировать вручную.

2.2 Extreme Points и Bottom-Left-Fill

Подход extreme points строит набор потенциальных точек размещения на основе уже поставленных коробов. После размещения очередного объекта появляются новые точки около его правой, дальней и верхней граней. Для каждой точки перебираются допустимые ориентации, затем проверяются границы паллеты, пересечения, опора и дополнительные ограничения. Эвристики на базе extreme points специально разработаны для трехмерной упаковки и позволяют резко сократить число рассматриваемых координат без перехода к грубой дискретизации пространства [7].

В исследуемом проекте greedy-решатель использует extreme points вместе с gravity drop. Фактически кандидатная точка задает координаты x и y , а координата z выбирается как минимальная высота, на которую короб может быть опущен без пересечения и с учетом штабелируемости нижних объектов. Такое поведение близко к эвристике Bottom-Left-Fill: предпочтение отдается более низким, левым и ближним позициям.

Практический смысл gravity drop состоит в том, что кандидатная точка не рассматривается как окончательная высота размещения. Если короб можно опустить ниже без пересечения с уже размещенными объектами, алгоритм делает это и тем самым уменьшает число висящих или неестественно высоких позиций. Для паллетизации это важно: низкий центр массы и заполнение нижних уровней обычно повышают устойчивость всей конструкции.

Набор extreme points после каждого размещения должен очищаться от заведомо бесполезных точек. В противном случае число кандидатов быстро растет, а решатель тратит большую часть времени на повторную проверку коор-

динат, которые уже заняты или находятся вне допустимой области. Типичные фильтры удаляют дубликаты, точки внутри уже размещенных коробов, точки за пределами паллеты и позиции, для которых невозможно поставить ни одну допустимую ориентацию. Такая фильтрация не меняет постановку задачи, но существенно влияет на скорость базового greedy и на качество начальных решений LNS.

Ограничение подхода extreme points связано с тем, что он сохраняет только локально естественные позиции. Если оптимальное решение требует временно оставить свободное место или разместить короб не в ближайшей нижней позиции, жадный декодер может не найти такую конфигурацию. Поэтому extreme points хорошо подходят для быстрого построения решений, но в сложных случаях требуют надстройки в виде локального поиска, рестартов или крупноокрестностной перестройки.

2.3 MaxRects и послойная укладка

Алгоритмы семейства MaxRects широко используются в двумерной упаковке прямоугольников. Они поддерживают список свободных прямоугольных областей и после размещения объекта разбивают занятое пространство на новые свободные области. В трехмерной паллетизации такой подход можно применять послойно: выбрать высоту слоя, упаковать в нем прямоугольные основания коробов, затем перейти к следующему уровню.

В проекте файл `solve_conditions.py` реализует MaxRects-подобный послойный решатель, адаптированный из идеи соревнования Packing Santa's Sleigh [10]. Метод быстро строит плотные слои и показывает высокую утилизацию объема, однако из-за послойной природы и сортировки по весу хуже контролирует хрупкость: на контрольном наборе средний fragility score равен 0.5198.

Главное преимущество послойной схемы заключается в уменьшении размерности промежуточной задачи. Вместо полного трехмерного поиска решатель фиксирует высоту слоя и решает двумерную задачу размещения оснований. Это делает метод быстрым и хорошо объяснимым: в каждом слое можно анализировать свободные прямоугольники, правила выбора позиции и причины отказа от размещения конкретного товара. Для инженерного прототипа такая прозрачность важна, поскольку позволяет быстро сравнивать изменения в

эвристике.

Слабое место MaxRects-подобной схемы – выбор высоты слоя. Если слой выбран по одному крупному коробу, рядом с ним могут оказаться объекты меньшей высоты, и над ними образуется неиспользуемый вертикальный объем. Если же слои часто менять, повышается фрагментация и появляются узкие свободные области, которые трудно заполнить последующими коробами. В реальной паллетизации эта проблема усиливается ограничениями `strict_upright` и `stackable`: не каждый товар можно повернуть или использовать как опору для следующего уровня.

Для рассматриваемого проекта MaxRects играет роль сильного быстрого метода. Он полезен не только как самостоятельный решатель, но и как ориентир для интерпретации результатов: если другой алгоритм проигрывает MaxRects по плотности, но выигрывает по хрупкости и полноте заказа, это показывает реальный компромисс между геометрическим заполнением и логистической пригодностью укладки.

2.4 Large Neighborhood Search

Large Neighborhood Search относится к метаэвристикам крупноокрестностного поиска [3]. В отличие от малых локальных перестановок, LNS на каждой итерации разрушает существенную часть текущего решения, а затем восстанавливает ее. Такой подход особенно полезен для задач упаковки, где качество сильно зависит от взаимного расположения групп объектов.

Типовой цикл LNS состоит из следующих этапов:

1. Построение начального решения;
2. Выбор фрагмента решения для удаления;
3. Восстановление частичного решения;
4. Оценка нового решения;
5. Принятие или отклонение нового состояния.

В исследуемой реализации LNS начинается с усиленной greedy-фазы, затем применяет несколько destroy-эвристик и beam search на этапе repair. Критерий принятия основан на simulated annealing, что позволяет иногда принимать ухудшения и выходить из локальных максимумов.

Смысл destroy-фазы состоит в том, чтобы временно отказаться от части уже построенной укладки. Удаление может быть случайным, направленным

на верхние уровни, связанным с локальным пространственным кластером или ориентированным на проблемные элементы, например на короба около нарушений хрупкости. Разные варианты разрушения дают разные типы окрестностей: одни позволяют улучшить плотность, другие исправляют плохую структуру поддержки, третьи помогают найти место для ранее неразмещенных товаров.

Repair-фаза восстанавливает решение, но уже в измененном контексте. Если удаленные короба вставляются в прежнем порядке и по тем же правилам, метод почти не отличается от повторного greedy. Поэтому в проекте используется beam search: на каждом шаге сохраняется несколько перспективных частичных вариантов, а не только один лучший. Это увеличивает вычислительные затраты, но позволяет не отбрасывать решение из-за локально спорного выбора, который может оказаться выгодным через несколько последующих размещений.

Принятие кандидата по simulated annealing добавляет управляемую стохастичность. Если принимать только улучшения, поиск быстро стабилизируется и перестает выходить из локального максимума. Если принимать ухудшения слишком часто, алгоритм теряет направленность и превращается в случайный перебор. Температура и скорость охлаждения задают баланс между этими режимами: в начале поиска допустимы более смелые перестройки, ближе к концу предпочтение отдается улучшению найденного лучшего решения.

Для задачи паллетизации LNS особенно удобен тем, что сохраняет валидатор как внешний модуль. Алгоритм может экспериментировать с разрушением и восстановлением, но итоговая оценка всегда проходит через единые правила: границы, пересечения, массу, опору, ориентации и хрупкость. Это снижает риск, что более сложный метод выиграет только за счет неявного ослабления ограничений.

2.5 Генетические алгоритмы и ML-подходы

Генетические алгоритмы применяются к 3D Bin Packing через кодирование порядка размещения, ориентаций или параметров декодера. Особь обычно не хранит координаты напрямую, а задает правила, по которым greedy-декодер строит укладку. Это уменьшает риск невалидных решений и позволяет использовать эволюционный поиск в пространстве перестановок.

Современные исследования также рассматривают гибридные схемы, где эволюционный поиск дополняется генеративными моделями [9]. В таком подходе генератор пытается породить перспективные перестановки или признаки решений, а дискриминатор помогает отличать их от слабых конфигураций. В проекте реализован облегченный вариант GAN+GA без тяжелых фреймворков машинного обучения: нейросетевые компоненты написаны на NumPy и используются для разнообразия популяции и модификации fitness.

Преимущество генетического алгоритма заключается в способности исследовать множество разных порядков размещения. Для 3D Bin Packing это существенно, потому что небольшое изменение первых элементов очереди может полностью изменить дальнейшую структуру укладки. Операторы скрещивания и мутации позволяют комбинировать удачные фрагменты разных решений, а турнирный отбор поддерживает давление в сторону более качественных особей.

Однако генетический подход чувствителен к способу кодирования. Если хромосома задает только порядок коробов, алгоритм может долго искать подходящие ориентации. Если она кодирует и порядок, и ориентации, пространство поиска становится больше, а случайные мутации чаще портят работоспособные комбинации. Поэтому в проекте выбран компромисс: генетическая часть управляет перестановкой и выбором ориентаций, а проверка геометрии остается в декодере и валидаторе.

ML-компонента в такой постановке не заменяет решатель целиком. Она используется как механизм формирования более разнообразных и потенциально перспективных кандидатов. Это важно для исследовательской работы: можно сравнить, дает ли простая обучаемая надстройка выигрыш по сравнению с классической метаэвристикой. В текущих результатах GAN+GA демонстрирует конкурентный уровень, но уступает LNS, что указывает на необходимость более богатого обучения или более тесной связи модели с геометрическими признаками задачи.

2.6 Точный перебор и branch-and-bound

Для малых экземпляров возможен точный или почти точный поиск. В проекте каталог `brute_force_vs_algo` содержит решатель branch-and-bound с backtracking и ограниченным перебором по extreme points. Для за-

дач до 12 объектов используется полный режим, для более крупных – limited discrepancy search с ограниченным числом отклонений от жадного порядка.

Такая реализация важна не как основной промышленный решатель, а как исследовательская точка сравнения. Она позволяет на малых наборах оценить, насколько эвристические решения близки к результатам более полного поиска. Сохраненные результаты показывают, что на малых задачах LNS близок к brute force по итоговому score, но работает существенно быстрее на более сложных наборах.

Branch-and-bound строит дерево вариантов, где очередной уровень соответствует выбору позиции и ориентации для следующего объекта. Если частичное решение уже не может привести к результату лучше текущего лучшего, соответствующая ветвь отсекается. В задачах упаковки такие оценки могут учитывать оставшийся объем, число неразмещенных объектов, допустимую высоту и грубую верхнюю границу будущего score. Даже простые отсечения сильно уменьшают перебор, но не устраняют экспоненциальный рост в худшем случае.

Limited discrepancy search использует другое наблюдение: жадный порядок часто близок к хорошему решению, но иногда требует нескольких исправлений. Вместо полного перебора всех перестановок метод ограничивает число отклонений от базового жадного выбора. Это дает промежуточный режим между greedy и полным branch-and-bound: качество повышается, но число ветвей остается управляемым для средних тестовых примеров.

В рамках данной работы точные и почти точные методы нужны прежде всего для верификации масштаба ошибки эвристик. Если LNS на малых задачах дает результат, близкий к перебору, это повышает доверие к его поведению на больших экземплярах, где точный расчет уже невозможен. Если же на малых наборах эвристика систематически проигрывает, значит проблема связана не только с размером задачи, но и с самой логикой построения решения.

2.7 Выводы по разделу

Рассмотренные методы различаются по компромиссу между скоростью и качеством. Greedy и MaxRects быстры и интерпретируемы, но ограничены локальностью или послойной структурой. LNS добавляет возможность перестройки уже найденной укладки и поэтому лучше подходит для сложных экземпляров. GAN+GA расширяет поиск за счет эволюционных и генеративных

механизмов, но требует настройки и не гарантирует превосходства над хорошо подобранной метаэвристикой. Brute force полезен для анализа малых задач, но не масштабируется на рабочие размеры.

Для проекта Smart 3D Packing наиболее важен не отдельный алгоритм сам по себе, а сопоставимость методов в единой постановке. Все подходы должны работать с одинаковым входным JSON, возвращать размещения в едином формате и проверяться одним валидатором. Только в этом случае сравнение score, утилизации объема, полноты заказа, хрупкости и времени выполнения отражает свойства алгоритмов, а не различия в правилах проверки.

3 РАЗРАБОТАННОЕ РЕШЕНИЕ SMART 3D PACKING

3.1 Структура репозитория

Репозиторий проекта организован как экспериментальная среда для исследования алгоритмов паллетизации. Основные каталоги и файлы приведены в таблице 1. Исходный код проекта размещен в репозитории GitHub [11].

Таблица 1 – Основные компоненты проекта

Компонент	Назначение
datasets	готовые single-pallet наборы задач в формате JSON
dataset_generation	генератор синтетических сценариев фуд-ритейла
solvers	основные single-pallet решатели: greedy, MaxRects, LNS, GAN+GA
validator.py	проверка жестких ограничений и расчет итоговой метрики
results	сохраненные результаты запусков решателей и рассчитанные метрики
brute_force_vs_algo	сравнение эвристик с branch-and-bound на малых задачах
multipallet	расширение на случай нескольких паллет
vizualizator	интерактивная визуализация укладок
benchmark_report.py, plot_benchmark.py	агрегация результатов и построение графиков

Общая схема работы проекта показана на рис. 1. Сначала формируется или загружается входной JSON, затем один из решателей строит укладку, валидатор проверяет корректность и вычисляет метрики, после чего результаты сохраняются и могут быть визуализированы или агрегированы в benchmark-отчет.

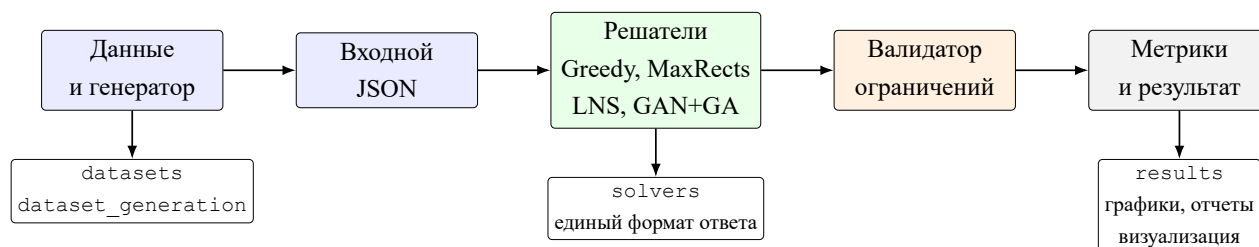


Рис. 1. Архитектура экспериментального контура проекта

3.2 Генерация данных

Генератор `dataset_generation/generator.py` использует набор товарных архетипов. Для каждого архетипа заданы базовые размеры, масса, признак вертикальной ориентации и хрупкость. При создании конкретной задачи размеры и масса варьируются небольшим случайным шумом, а количество экземпляров выбирается из диапазона.

В проекте реализованы сценарии:

- `heavy_water` – большое число тяжелых упаковок воды и сахара;
- `fragile_tower` – комбинация бананов, чипсов и яиц с хрупкими товарами;
- `liquid_tetris` – вода, вино и консервы с ограничениями ориентации;
- `high_count_mixed` – много разнотипных товаров;
- `heavy_fragile_mix` – тяжелые и хрупкие товары в одном заказе;
- `random_mixed` – случайная смесь нескольких архетипов.

Состав датасетов приведен в таблице 2. Эти данные были получены автоматическим анализом файлов из каталога `datasets`.

Таблица 2 – Характеристики используемых датасетов

Датасет	Задач	Ср. объектов	Мин.	Макс.	Ср. SKU
<code>dataset_100.json</code>	100	94.6	35	194	3.4
<code>dataset_150.json</code>	150	94.9	36	185	3.3
<code>dataset_200_2.json</code>	200	94.7	33	194	3.4
<code>dataset_benchmark.json</code>	200	140.0	31	316	3.7

Сценарный состав сгенерированных наборов задач показан на рис. 2.

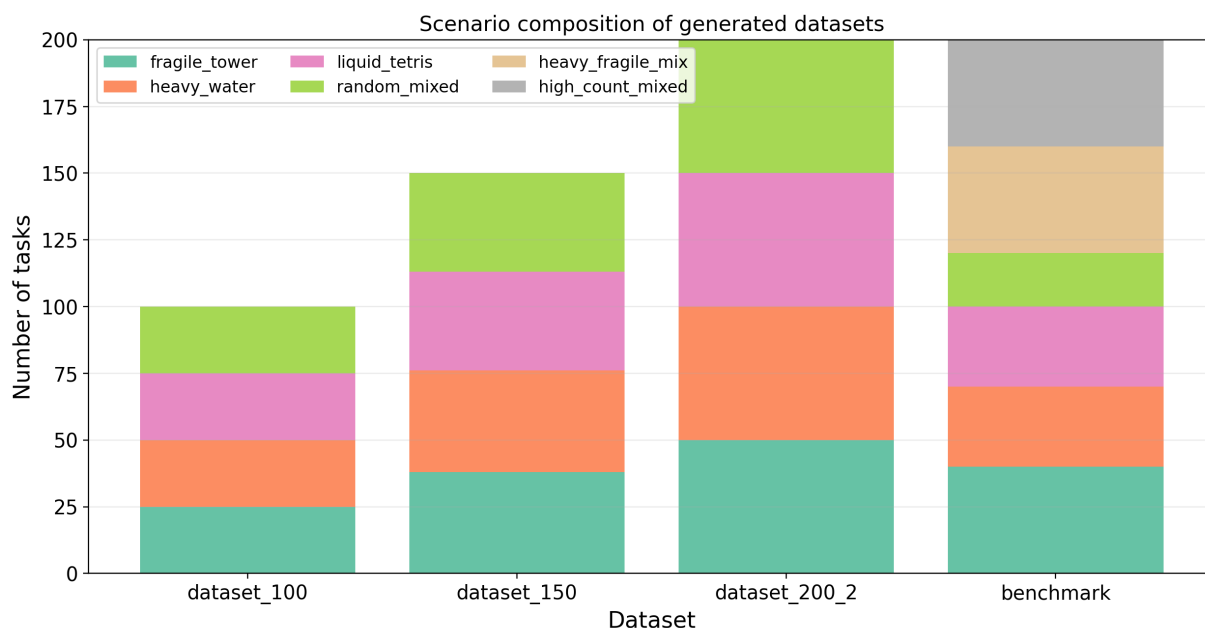


Рис. 2. Сценарный состав сгенерированных наборов задач

3.3 Валидатор и единая оценка

Файл `validator.py` является центральным элементом экспериментальной методологии. Он не зависит от конкретного решателя и выполняет одинаковые проверки для всех ответов. Сначала валидатор восстанавливает геометрию размещенных коробов по координатам и размерам, затем проверяет anti-cheat условие: фактические размеры должны быть перестановкой исходных.

После этого последовательно проверяются масса, границы паллеты, ограничение `strict_upright`, трехмерные пересечения и опора. Только валидные решения получают метрики. Такой подход исключает ситуацию, когда один метод оценивается по более мягким правилам, чем другой.

3.4 Greedy-решатель

Файл `solvers/greedy_algorithm.py` реализует последовательный алгоритм на `extreme points`. Количество каждого SKU разворачивается в список отдельных экземпляров. Затем объекты сортируются по объему, причем хрупкие и нештабелируемые товары получают специальный приоритет, чтобы уменьшить риск плохой структуры укладки.

Для каждого объекта перебираются допустимые повороты. Функция `get_rotations` возвращает ориентации из множества LWH, LHW, WLH, WHL,

HLW, HWL; для `strict_upright` остаются только повороты с исходной высотой. Проверки пересечений, опоры и хрупкости ускорены пространственной сеткой `PlacedGrid`, которая ограничивает число уже размещенных коробов, участвующих в геометрических проверках.

Преимущество `greedy` – скорость и простота. На всех сохраненных `single-pallet` наборах средний временной балл равен 1.0. Недостаток – невозможность перестроить ранние решения, что особенно заметно на `dataset_benchmark.json`: средний `item coverage` снижается до 0.4906.

3.5 MaxRects-решатель

Файл `solvers/solve_conditions.py` реализует послойный подход. Для каждого слоя выбирается высота-кандидат, после чего основания коробов упаковываются двумерным MaxRects-подобным алгоритмом. Метод сортирует товары по массе и объему, пытается заполнить слой, затем переходит к следующей высоте.

Этот решатель очень быстр: среднее время на контрольном наборе составляет около 52 мс. Он достигает высокой утилизации объема, но заметно проигрывает по хрупкости. Такое поведение объясняется тем, что послойная плотность и сортировка по весу не всегда согласуются с правилом размещения хрупких товаров верхним слоем.

3.6 LNS-решатель

Файл `solvers/lms_solver.py` содержит наиболее сильный решатель проекта. Схема его работы показана на рис. 3.

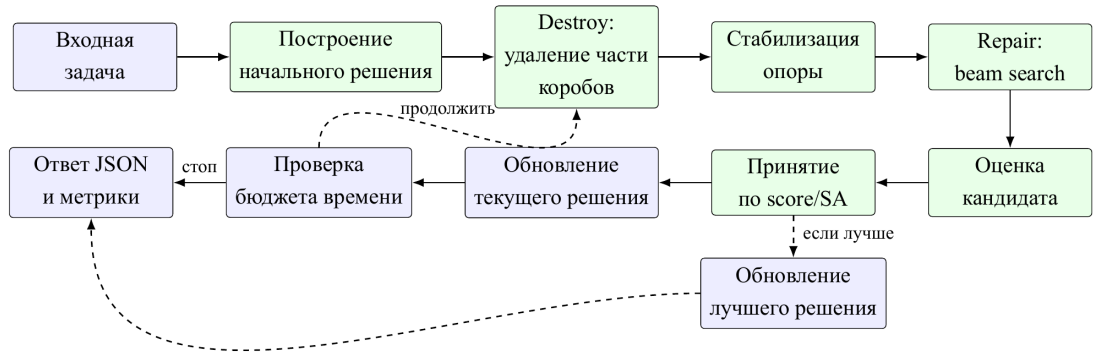


Рис. 3. Основные этапы LNS-решателя

Сначала LNS строит начальное решение. В отличие от одиночного greedy-запуска, используется несколько детерминированных порядков и случайные рестарты в пределах бюджета `greedy_phase_s`. Затем запускается основной цикл `destroy/repair`. В реализации присутствуют `destroy`-эвристики:

- Случайное удаление размещенных коробов;
- Удаление верхних коробов;
- Удаление пространственного кластера;
- Удаление коробов, связанных с нарушениями хрупкости;
- Удаление коробов, близких по объему к непо помещенным;
- Полный рестарт;
- Удаление малых размещенных коробов.

После разрушения вызывается стабилизация частичного решения: если некоторые короба потеряли достаточную опору, они также удаляются. Этап `repair` основан на `beam search`. Для частичных состояний применяется оценка по формуле (6):

$$\text{partial_score} = 0.50U_{\text{vol}} + 0.30C_{\text{items}} - 0.005n_{\text{frag_viol}}, \quad (6)$$

которая ориентирует восстановление на объем, покрытие и умеренный штраф за потенциальные нарушения хрупкости.

Ключевые параметры LNS приведены в таблице 3.

Таблица 3 – Основные параметры LNS-конфигурации

Параметр	Значение	Назначение
time_budget_s	4.8	общий бюджет времени
greedy_phase_s	1.5	стартовая greedy-фаза
destroy_k_fraction	0.15	доля удаляемых коробов
destroy_k_min/max	3 / 15	границы размера разрушения
beam_width	8	ширина beam search
repair_queue_limit	50	максимальная очередь восстановления
repair_restarts	2	дополнительные repair-рестарты
early_stop_patience	25	остановка при отсутствии улучшений
ils_kicks	5	число ILS-перезапусков
sa_initial_temp	0.015	начальная температура simulated annealing
sa_cooling_rate	0.92	скорость охлаждения

3.7 GAN+GA-решатель

Файл `solvers/gan_ga_solver.py` реализует гибрид генетического алгоритма и простой генеративно-сопоставительной модели. Хромосома состоит из перестановки объектов и списка выборов ориентаций. Декодер строит укладку greedy-методом с ограниченным числом extreme points.

Генетическая часть использует турнирный отбор, order crossover, скрещивание ориентаций и мутации перестановки. GAN-компонента реализована на NumPy: генератор создает приоритеты объектов, преобразуемые в перестановку, а дискриминатор оценивает похожесть решения на элитные особи. Эта оценка добавляется к fitness с малым коэффициентом.

Метод интересен как ML-ориентированная альтернатива, но в текущих экспериментах уступает LNS по среднему score на общих наборах. При этом GAN+GA сохраняет высокий fragility score и демонстрирует конкурентную утилизацию объема.

3.8 Визуализация

Каталог `vizualizator` содержит средства просмотра результатов. Визуализатор читает входной JSON и JSON с размещениями, строит трехмерное представление паллеты и позволяет анализировать слои. Для дипломной работы на основе сохраненного результата LNS построен пример укладки на рис. 4.

Визуализация используется не как отдельная метрика, а как инструмент проверки интерпретации результатов. По числовым значениям `score` можно увидеть, какой метод лучше заполнил паллету или разместил больше коробов, но трудно понять, почему возникло отличие. Трехмерный просмотр помогает обнаружить характерные ситуации: пустые вертикальные полости после послойной укладки, слишком раннее размещение крупного короба, слабое использование верхних уровней или неудачное соседство хрупких и тяжелых товаров.

Для анализа LNS визуализация особенно полезна, потому что метод многократно перестраивает часть решения. Если итоговая укладка оказывается лучше `greedy`, важно проверить, что улучшение действительно связано с более рациональным размещением, а не только с изменением порядка неразмещенных объектов. Просмотр слоев показывает, как `gerair`-фаза заполняет свободные области и сохраняет опору для верхних коробов.

В текущей реализации визуализатор отделен от решателей и валидатора. Такое разделение удобно для экспериментов: один и тот же результат можно сначала проверить формально, затем открыть в графическом виде и сопоставить с агрегированными метриками. Это также снижает риск ручной ошибки при подготовке иллюстраций для отчета, поскольку изображение строится по тому же JSON-ответу, который используется при расчете `benchmark`-таблиц.

Example LNS packing for dataset_100 first task

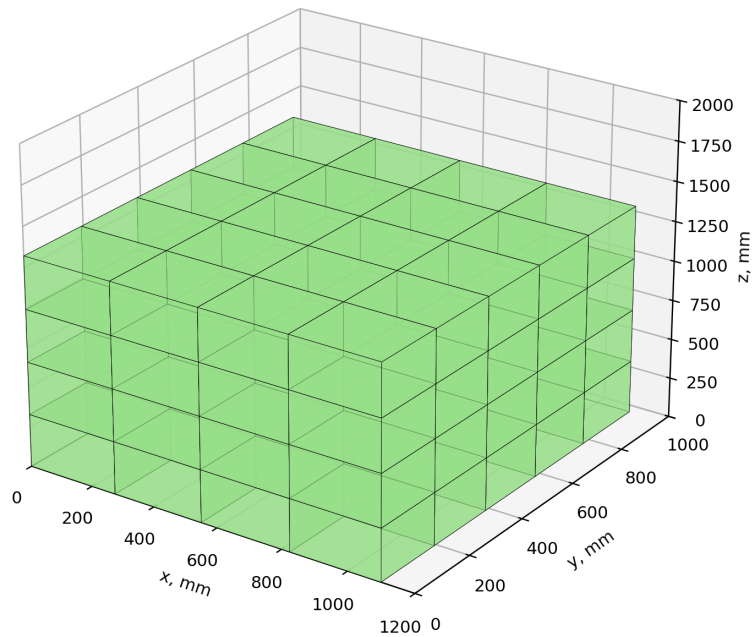


Рис. 4. Пример укладки, построенной LNS для первой задачи dataset_100.json

3.9 Выводы по разделу

Проект представляет собой не отдельный алгоритм, а полноценный исследовательский стенд. Его сильная сторона – единый формат данных, общий валидатор и сохраненные результаты для нескольких решателей. На уровне реализации наиболее развитым методом является LNS: он использует greedy как стартовую эвристику, но дополняет ее разрушением и восстановлением фрагментов решения, адаптивными эвристиками и simulated annealing.

4 ЭКСПЕРИМЕНТЫ, МЕТРИКИ И АНАЛИЗ РЕЗУЛЬТАТОВ

4.1 Методика эксперимента

Экспериментальная часть основана на уже сохраненных результатах из каталога `results`. Для каждого решателя и датасета в JSON-файлах содержатся ответы по отдельным задачам, рассчитанные метрики и время решения. Такой подход выбран намеренно: он исключает повторную случайную генерацию результатов и позволяет опираться на фактические данные проекта.

При сравнении эвристических методов особенно важно использовать единый протокол оценки. Если один алгоритм проверяется более мягким валидатором, а другой – более строгим, итоговое сравнение теряет смысл. В данном проекте все результаты проходят через `validator.py`, поэтому различия в `score` отражают именно свойства построенных укладок. Это также упрощает воспроизводимость: любой сохраненный JSON-ответ можно повторно проверить тем же валидатором и получить те же компоненты метрики.

Сохраненные результаты рассматриваются как экспериментальная выборка, а не как единичные демонстрационные примеры. Каждый датасет содержит множество задач с разным составом товаров, поэтому средние значения позволяют оценивать устойчивость метода к изменению входных данных. При этом отдельные наборы отличаются сложностью: `dataset_benchmark.json` содержит больше объектов в задаче и более разнообразный сценарный состав, поэтому он используется как наиболее показательный контрольный набор.

В основном сравнении используются четыре `single-pallet` метода:

- `greedy_algorithm`;
- `conditions` – `MaxRects`-подобный послойный решатель;
- `gan_ga_solver`;
- `lns_solver`.

Общими для всех четырех решателей являются три набора: `dataset_100.json`, `dataset_200_2.json` и `dataset_benchmark.json`. Для GAN+GA дополнительно есть результат на `dataset_150.json`, но он не используется в основном ранжировании, чтобы сравнение было симметричным.

Симметричность сравнения важна из-за того, что итоговый score усредняется по задачам внутри набора. Если включить датасет, на котором присутствует только один решатель, средний результат нельзя будет напрямую сопоставлять с остальными методами. Поэтому дополнительный результат GAN+GA полезен для анализа поведения метода, но не должен менять основную таблицу ранжирования.

4.2 Сводные результаты

Средний итоговый score на общих наборах приведен в таблице 4 и на рис. 5.

Таблица 4 – Средний итоговый score на общих наборах

Метод	dataset_100	dataset_200_2	dataset_benchmark
Greedy	0.6842	0.7120	0.6549
MaxRects	0.7104	0.7117	0.7124
GAN+GA	0.7012	0.7327	0.7161
LNS	0.7111	0.7434	0.7298

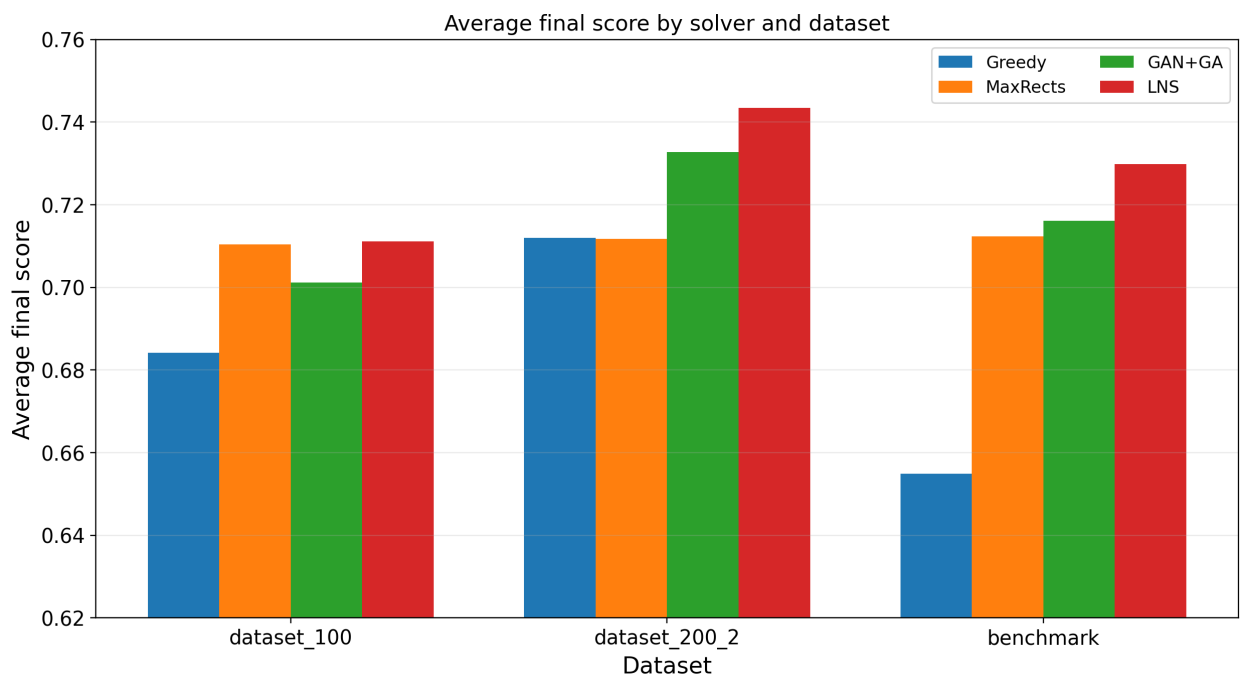


Рис. 5. Средний итоговый score по решателям и датасетам

LNS показывает лучший результат на всех трех общих наборах. На dataset_100.json его преимущество над MaxRects минимально, что объясняется меньшей сложностью задач: быстрый послойный метод уже стро-

ит достаточно сильные решения. На `dataset_200_2.json` LNS заметно обгоняет остальные методы за счет лучшей полноты размещения. На `dataset_benchmark.json`, где среднее число объектов выше и состав сценариев шире, преимущество LNS сохраняется.

Интерпретация этой таблицы показывает, что выигрыш LNS не является случайным эффектом одного датасета. Метод не всегда максимизирует отдельную компоненту, но стабильно удерживает высокий итоговый баланс. Для практической паллетизации такое поведение предпочтительнее, чем резкий выигрыш по одной характеристике при сильном проигрыше по другой. Например, алгоритм с максимальной утилизацией объема может оказаться менее полезным, если он часто размещает тяжелые товары над хрупкими или оставляет большую долю заказа вне паллеты.

Также важно учитывать, что различие между методами зависит от сложности набора. На простых задачах сильный baseline может быть близок к LNS, потому что пространство хороших решений достаточно широкое. На более насыщенных задачах ранние ошибки greedy и послойных методов становятся дороже: один неудачно выбранный нижний слой ограничивает все последующие уровни. В этих случаях возможность разрушить и восстановить часть укладки дает LNS преимущество.

4.3 Компоненты метрики

Для интерпретации результатов рассмотрим компоненты score на контрольном наборе `dataset_benchmark.json`. Данные приведены в таблице 5 и на рис. 6.

Таблица 5 – Компоненты метрики на `dataset_benchmark.json`

Метод	Score	Vol.	Coverage	Fragility	Time
Greedy	0.6549	0.6155	0.4906	1.0000	1.0000
MaxRects	0.7124	0.7513	0.6160	0.5198	1.0000
GAN+GA	0.7161	0.7156	0.5540	1.0000	0.9205
LNS	0.7298	0.7172	0.6036	0.9833	0.9175

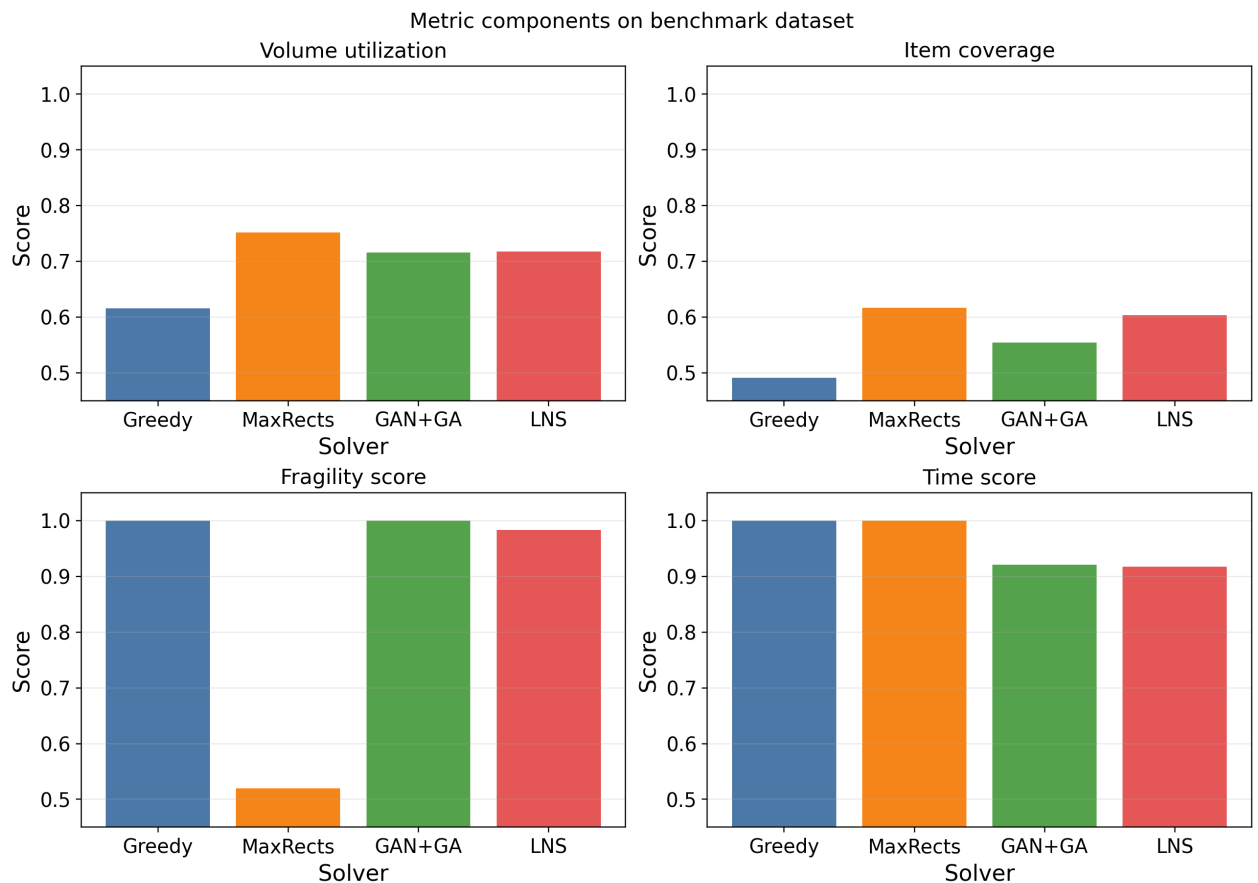


Рис. 6. Компоненты score на контрольном наборе

Greedy сохраняет идеальные значения по хрупкости и времени, но значительно уступает по покрытию заказа. MaxRects достигает максимальной утилизации объема и максимального покрытия на контрольном наборе, однако теряет почти половину балла за хрупкость. GAN+GA хорошо соблюдает хрупкость и заметно улучшает объем по сравнению с greedy, но уступает LNS по покрытию.

LNS не максимизирует отдельную компоненту любой ценой. Его сила заключается в балансе: утилизация объема близка к GAN+GA, покрытие близко к MaxRects, хрупкость остается высокой, а временной балл остается приемлемым. Именно этот баланс дает лучший итоговый score.

Компонентный анализ показывает, почему итоговая метрика полезнее одного показателя заполнения объема. MaxRects достигает наибольшей утилизации и покрытия на контрольном наборе, но сильная потеря по хрупкости снижает итоговую оценку. Greedy, наоборот, сохраняет идеальную хрупкость и скорость, но слишком часто оставляет часть заказа неразмещенной. LNS занимает промежуточное положение по отдельным компонентам, но именно поэтому оказывается наиболее устойчивым по суммарному критерию.

Такой результат согласуется с логикой выбранной метрики. В реаль-

ной задаче паллетизации нельзя оптимизировать только геометрию, игнорируя свойства товара. Укладка должна быть плотной, но не должна создавать чрезмерный риск повреждения, а время расчета должно оставаться приемлемым для использования в операционном контуре. Поэтому метод, который немного уступает по объему, но лучше соблюдает остальные требования, может получить более высокий итоговый score.

4.4 Время выполнения

Среднее фактическое время решения приведено в таблице 6. Все методы на основных наборах остаются в пределах практически приемлемого времени. Greedy и MaxRects работают быстрее всех, LNS и GAN+GA тратят около секунды на контрольном наборе.

Таблица 6 – Среднее время решения, мс

Метод	dataset_100	dataset_200_2	dataset_benchmark
Greedy	114.8	118.5	139.0
MaxRects	15.1	15.6	52.0
GAN+GA	795.3	770.4	1080.6
LNS	474.3	523.6	1062.6

Увеличение времени LNS оправдано ростом основных компонент качества. На контрольном наборе LNS теряет часть временного балла, но выигрывает по итоговой оценке. Это соответствует назначению метаэвристики: использовать дополнительное время не на случайное усложнение, а на улучшение структуры укладки.

С точки зрения прикладного применения абсолютное время LNS остается небольшим: на контрольном наборе оно измеряется примерно одной секундой. Для пакетного планирования паллет или предварительного расчета заказов такой уровень затрат обычно приемлем. Если же система должна работать строго в интерактивном режиме, можно уменьшить бюджет LNS и использовать greedy или MaxRects как быстрый fallback, сохраняя единый формат ответа и валидации.

При этом время выполнения следует рассматривать вместе с размером задачи и требуемым качеством ответа. Для небольших заказов выигрыш сложной метаэвристики может быть несущественным, поскольку быстрый решатель

уже находит близкое по качеству решение. Для насыщенных заказов с большим числом коробов дополнительная секунда расчета может окупаться за счет более полного размещения и меньшего числа переносов товара на следующую паллету. Поэтому практическая система может выбирать метод адаптивно: запускать быстрый baseline для простых случаев и включать LNS для задач, где требуется улучшить плотность или покрытие.

4.5 Сравнение LNS и Greedy

В рамках НИР отдельно исследовался LNS как развитие базового greedy. Для этой пары доступны графики, построенные из сохраненных результатов. На рис. 7 показано, что LNS стабильно превосходит greedy по среднему score на трех наборах.

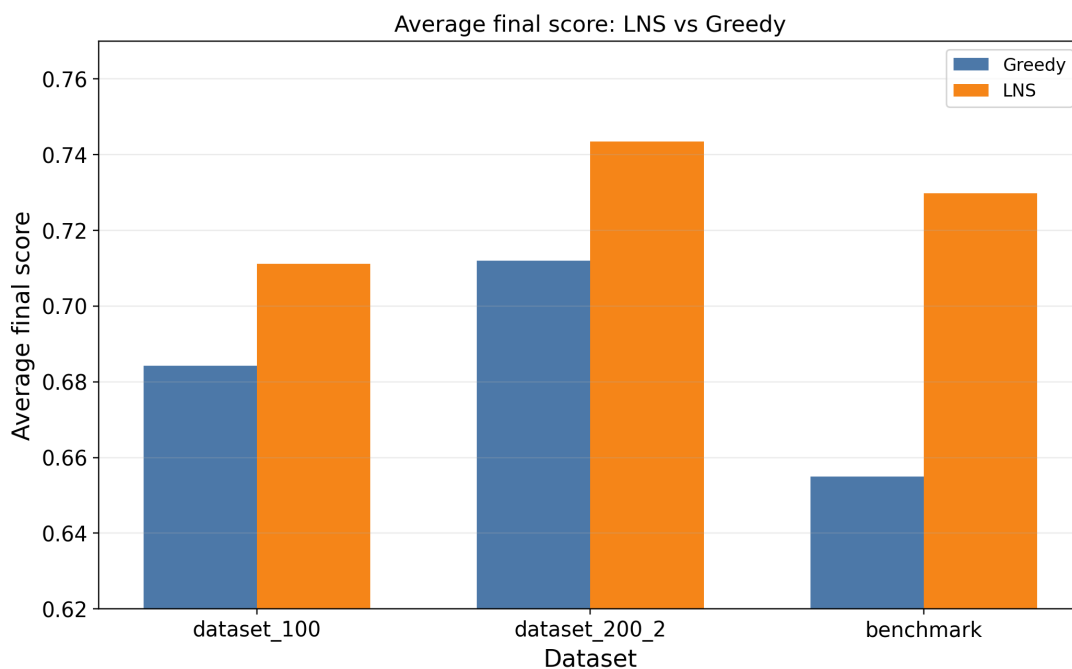


Рис. 7. Сравнение среднего score для LNS и greedy

Компонентное сравнение на рис. 8 показывает, что главный вклад LNS дает по утилизации объема и покрытию заказа. Небольшое ухудшение по времени и хрупкости не перекрывает этот выигрыш.

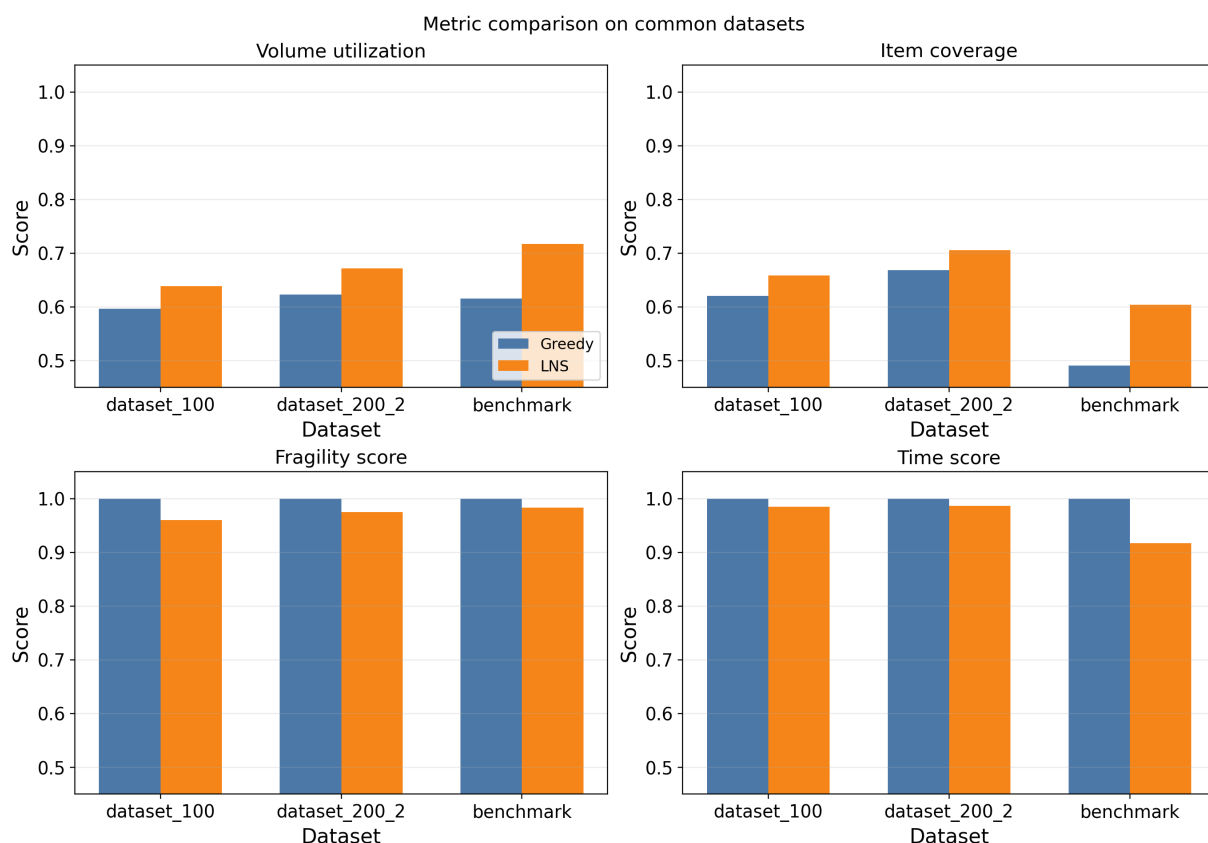


Рис. 8. Сравнение компонент метрики для LNS и greedy

4.6 Сравнение с brute force на малых задачах

Каталог `brute_force_vs_algo` содержит отдельные эксперименты для малых и средних наборов. Их цель – оценить поведение LNS рядом с более полным поиском. Результаты приведены в таблице 7.

Таблица 7 – Сравнение brute force и LNS на специальных наборах

Набор	Метод	Score	Coverage	Время, мс
small	Brute force	0.5563	0.9897	19.5
small	LNS	0.5602	1.0000	28.3
medium	Brute force	0.5681	0.9921	10440.7
medium	LNS	0.5780	1.0000	72.5
hard	Brute force / LDS	0.5886	0.9364	8815.0
hard	LNS	0.6501	1.0000	243.2

На этих наборах LNS не только сопоставим с перебором, но и превосходит сохраненные результаты brute force/LDS по итоговой метрике. Важно, что для medium и hard время перебора резко возрастает, тогда как LNS остается в

пределах сотен миллисекунд. Это подтверждает практическую ценность метазвристического подхода.

4.7 Многопаллетное расширение

В каталоге `multipallet` реализовано расширение на несколько паллет. Эта постановка ближе к реальной логистике: если весь заказ нельзя разместить на одной паллете, требуется распределить оставшиеся товары на новые паллеты. В проекте есть `single-pallet` и `multi-pallet` решения для `greedy` и `LNS`.

В рамках данной ВКР многопаллетный режим рассматривается как направление развития, поскольку основная метрика и основные `benchmark`-результаты сосредоточены на `single-pallet` постановке. Тем не менее наличие отдельного каталога `multipallet` показывает, что архитектура проекта допускает переход от локальной задачи плотной укладки к задаче распределения заказа между несколькими контейнерами.

Многопаллетная постановка меняет смысл оптимизации. В `single-pallet` варианте часть товара может остаться неразмещенной, и это отражается в компоненте покрытия. В `multi-pallet` режиме требуется решить, сколько паллет использовать и как распределить товары между ними. Тогда к текущим критериям добавляется стоимость дополнительной паллеты, а также могут появиться требования группировки товаров по магазинам, маршрутам или температурным зонам. Поэтому имеющееся расширение следует рассматривать как основу, которую нужно дополнить отдельной метрикой и отдельными экспериментами.

4.8 Анализ ошибок и ограничений

Анализ кода и результатов позволяет выделить несколько ограничений текущей реализации.

Во-первых, модель устойчивости приближенная. Критерий 60% опоры учитывает площадь контакта, но не анализирует центр масс, распределение давления, трение и деформации упаковки. Поэтому решение может быть валидным по формальному критерию, но требовать дополнительной инженерной проверки перед промышленным применением.

Во-вторых, синтетические данные не покрывают всего разнообразия реальных заказов. В генераторе есть полезные сценарии `фуд-ритейла`, но нет, на-

пример, температурных зон, запрета соседства, приоритетов выгрузки, ограничений по слоям и требований к группировке товаров по магазинам.

В-третьих, GAN+GA является облегченной исследовательской реализацией. Он использует NumPy-модель вместо полноценного обучения на большом корпусе решений. Поэтому текущие результаты не следует трактовать как окончательный предел ML-подходов к задаче.

В-четвертых, benchmark хранит средние значения, но не содержит статистического анализа устойчивости стохастических методов по нескольким независимым запускам на каждом датасете. Для LNS и GAN+GA в дальнейшем полезно оценивать дисперсию результата.

Еще одно ограничение связано с синтетической природой основной части данных. Генератор позволяет управлять сценариями и получать воспроизводимые наборы, однако реальные складские заказы могут иметь дополнительные зависимости: совместимость товарных групп, требования к последовательности выгрузки, ограничения по температуре, запрет соседства некоторых позиций и нестандартные размеры упаковок. Поэтому полученные результаты показывают качество методов в моделируемой постановке, но для промышленного внедрения их следует дополнительно проверять на реальных данных конкретной логистической системы.

Кроме того, сохраненные benchmark-результаты фиксируют итоговые средние значения, но не всегда показывают распределение ошибок по отдельным задачам. Для дальнейшего анализа полезно выделять группы экземпляров, на которых метод чаще всего проигрывает: заказы с большим числом мелких коробов, смеси тяжелых и хрупких товаров, задачи с ограниченными поворотами или случаи почти полного заполнения паллеты. Такая детализация поможет не только улучшить средний score, но и понять, какие именно ограничения требуют новых эвристик.

4.9 Направления развития

Наиболее перспективные направления дальнейшей работы:

1. Улучшить физическую модель устойчивости с учетом центра масс и распределения веса;
2. Расширить генератор данных реальными бизнес-ограничениями;

3. Добавить статистический протокол повторных запусков стохастических методов;
4. Развить LNS за счет адаптивного выбора destroy/repair-операторов;
5. Исследовать reinforcement learning или imitation learning для выбора порядка размещения;
6. Развить многопаллетную постановку с метрикой минимизации числа паллет;
7. Добавить автоматическую генерацию HTML-отчета с интерактивной 3D-визуализацией.

4.10 Выводы по разделу

Экспериментальные результаты показывают, что лучшим single-pallet методом текущего проекта является LNS. Он стабильно превосходит greedy, MaxRects и GAN+GA по среднему итоговому score на общих наборах. Причина преимуществ не в максимизации одной компоненты, а в сбалансированном улучшении объема, покрытия, хрупкости и времени. Greedy остается полезным быстрым baseline, MaxRects – сильным послойным решателем по плотности, GAN+GA – перспективным исследовательским направлением, а brute force – инструментом проверки малых экземпляров.

ЗАКЛЮЧЕНИЕ

В выпускной квалификационной работе была исследована задача Smart 3D Packing, представляющая собой прикладную постановку 3D Bin Packing для укладки коробов на паллету. В отличие от абстрактной геометрической задачи, рассматриваемая постановка учитывает ограничения, характерные для складской логистики: габариты паллеты, грузоподъемность, допустимые повороты, опору, хрупкость, штабелируемость и время расчета.

В ходе работы были изучены условие задачи, материалы НИР и курсовой работы, требования к оформлению ВКР, примеры прошлых дипломов и исходный код проекта. По результатам анализа восстановлен pipeline решения: генерация или загрузка JSON-датасета, запуск одного из решателей, валидация жестких ограничений, расчет метрик, сохранение результатов и визуализация укладки.

Были рассмотрены реализованные методы: greedy на основе extreme points и gravity drop, послойный MaxRects-решатель, Large Neighborhood Search, гибрид GAN+GA, branch-and-bound для малых задач и многопаллетное расширение. Для каждого метода описаны основная идея, роль в проекте, преимущества и ограничения.

Отдельное внимание было уделено тому, что все методы сравниваются в единой постановке. Входные данные имеют общий JSON-формат, результаты решателей приводятся к единой структуре ответа, а проверка выполняется одним валидатором. Благодаря этому различия между алгоритмами интерпретируются как различия в качестве построенной укладки, а не как следствие разных правил проверки или разных форматов результатов.

Экспериментальная часть основана на сохраненных результатах репозитория. Сравнение на общих single-pallet наборах показало, что LNS является наиболее эффективным методом по итоговой метрике: 0.7111 на наборе из 100 задач, 0.7434 на наборе из 200 задач и 0.7298 на контрольном наборе. Его преимущество связано с тем, что метод умеет перестраивать уже найденную укладку, исправляя локально неудачные ранние решения greedy.

Greedy показал высокую скорость и идеальный временной балл, но уступил по утилизации объема и покрытию заказа. MaxRects достиг высокой плотности, но потерял качество по хрупкости. GAN+GA подтвердил перспективность гибридного ML-ориентированного поиска, однако в текущей реализации

не превзошел LNS. Brute force оказался полезен для малых задач, но плохо масштабируется на более крупные экземпляры.

Проведенный анализ также показал ограничения текущего этапа работы. Эксперименты основаны преимущественно на синтетических данных, физическая модель устойчивости упрощена до проверки площади опоры, а стохастические методы требуют дополнительной оценки разброса результатов по нескольким независимым запускам. Эти ограничения не отменяют полученных выводов, но задают направления, в которых проект может быть усилен.

Практическая ценность полученного решения состоит в том, что проект уже содержит не только отдельные алгоритмы, но и полный контур проверки: генерацию задач, запуск решателей, расчет метрик, хранение результатов и визуализацию. Такая организация позволяет постепенно улучшать отдельные методы, не меняя постановку эксперимента целиком. Например, можно заменить destroy-оператор в LNS, расширить генератор данных или уточнить правило хрупкости, после чего проверить изменение результата тем же benchmark-процессом.

Для дальнейшего развития наиболее важной является проверка на данных, близких к реальным складским заказам. Синтетические сценарии полезны для управляемого сравнения, но промышленная эксплуатация требует учитывать дополнительные ограничения: совместимость товарных групп, требования к разгрузке, температурные зоны и особенности конкретной упаковки. Поэтому следующим этапом может стать сбор обезличенных заказов или создание генератора, параметры которого настраиваются по статистике реального склада.

Содержательно работа показывает, что для 3D Bin Packing в логистической постановке наиболее продуктивен комбинированный подход. Быстрые эвристики нужны для построения стартовых решений и контроля времени, послойные методы дают сильную геометрическую базу, метаэвристики позволяют исправлять локально неудачные решения, а ML-ориентированные методы могут использоваться как источник новых порядков и признаков для поиска. В такой системе каждый подход занимает свою роль, поэтому дальнейшее развитие проекта целесообразно строить не как замену всех методов одним универсальным алгоритмом, а как улучшение общего экспериментального контура.

Таким образом, цель работы достигнута: выполнен анализ предметной области, формализована постановка задачи, изучена реализация проекта, опи-

саны алгоритмы и проведено сравнение методов. Наиболее рациональным основным методом для текущей single-pallet постановки является LNS, а дальнейшее развитие проекта целесообразно связывать с усилением его destroy/repair-операторов, улучшением физической модели устойчивости, расширением генератора данных и развитием многопаллетного режима.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Martello S., Pisinger D., Vigo D. The Three-Dimensional Bin Packing Problem // *Operations Research*. 2000. Vol. 48, No. 2. P. 256–267. DOI: <https://doi.org/10.1287/opre.48.2.256.12386>.
2. Faroe O., Pisinger D., Zachariasen M. Guided Local Search for the Three-Dimensional Bin-Packing Problem // *INFORMS Journal on Computing*. 2003. Vol. 15, No. 3. P. 267–283.
3. Pisinger D., Ropke S. Large Neighborhood Search // *Handbook of Metaheuristics*. 2nd ed. Springer, 2010. P. 399–420.
4. Dyckhoff H., Finke U. Cutting and Packing in Production and Distribution: Typology and Bibliography. Heidelberg: Physica-Verlag, 1992. 248 p.
5. Coffman E. G., Garey M. R., Johnson D. S. Approximation Algorithms for Bin Packing: A Survey // *Approximation Algorithms for NP-Hard Problems*. Boston: PWS Publishing, 1997. P. 46–93.
6. Lodi A., Martello S., Vigo D. Recent Advances on Two-Dimensional Bin Packing Problems // *Discrete Applied Mathematics*. 2002. Vol. 123. P. 379–396.
7. Crainic T. G., Perboli G., Tadei R. Extreme Point-Based Heuristics for Three-Dimensional Bin Packing // *INFORMS Journal on Computing*. 2008. Vol. 20, No. 3. P. 368–384.
8. Gendreau M., Iori M., Laporte G., Martello S. A Tabu Search Algorithm for a Routing and Container Loading Problem // *Transportation Science*. 2006. Vol. 40, No. 3. P. 342–350.
9. Zhang Y., Wang X., Liu Z. A Novel Approach for Solving 3D Bin Packing Problem by Integrating a Generative Adversarial Network with a Genetic Algorithm // *Scientific Reports*. 2024. DOI: <https://doi.org/10.1038/s41598-024-56699-7>.
10. Packing Santa's Sleigh // Kaggle: соревнование по задаче трехмерной упаковки [Электронный ресурс]. URL: <https://www.kaggle.com/competitions/packing-santas-sleigh/overview> (дата обращения: 27.04.2026).

11. DrogoZZZZ. 3D Bin Packing [Электронный ресурс]: GitHub-репозиторий проекта. URL: https://github.com/DrogoZZZZ/3D_bin_packing (дата обращения: 26.05.2026).

ПРИЛОЖЕНИЕ А

Сводная таблица benchmark-результатов

Решатель	Датасет	Score	Vol.	Coverage	Time, мс
Greedy	dataset_100. json	0.6842	0.5961	0.6204	114.8
Greedy	dataset_200_2. json	0.7120	0.6228	0.6686	118.5
Greedy	dataset_ benchmark.json	0.6549	0.6155	0.4906	139.0
MaxRects	dataset_100. json	0.7104	0.7005	0.6825	15.1
MaxRects	dataset_200_2. json	0.7117	0.7000	0.6809	15.6
MaxRects	dataset_ benchmark.json	0.7124	0.7513	0.6160	52.0
GAN+GA	dataset_100. json	0.7012	0.6278	0.6271	795.3
GAN+GA	dataset_150. json	0.7262	0.6571	0.6589	769.3
GAN+GA	dataset_200_2. json	0.7327	0.6700	0.6591	770.4
GAN+GA	dataset_ benchmark.json	0.7161	0.7156	0.5540	1080.6
LNS	dataset_100. json	0.7111	0.6384	0.6580	474.3
LNS	dataset_200_2. json	0.7434	0.6712	0.7055	523.6
LNS	dataset_ benchmark.json	0.7298	0.7172	0.6036	1062.6

ПРИЛОЖЕНИЕ Б

Ключевые файлы реализации

Таблица 9 – Файлы, использованные при анализе реализации

Файл	Содержание
validator.py	проверка границ, пересечений, опоры, массы, ориентации и расчет score
solvers/greedy_algorithm.py	greedy на extreme points, gravity drop, spatial hash grid
solvers/solve_conditions.py	послойный MaxRects-подобный решатель
solvers/lms_solver.py	LMS с destroy/repair, beam search, simulated annealing и ILS-рестартами
solvers/gan_ga_solver.py	генетический алгоритм с NumPy-GAN компонентой
dataset_generation/generator.py	архетипы товаров и сценарии генерации задач
brute_force_vs_algo/brute_force.py	branch-and-bound и limited discrepancy search для малых задач
multipallet/solver.py, multipallet/lms_solver.py	многопаллетные расширения
vizualizator/ visualizator.py	просмотр single-pallet укладки
benchmark_report.py	агрегация JSON-результатов в CSV